

GERÊNCIA E QUALIDADE DE SOFTWARE

Unidade I — Dimensões da Qualidade & Técnicas de Revisão (Parte I)

Aula 3 · 01/09/2026 · 2h

Profa. Ana Carolina Gondim Inocêncio

Roteiro da Aula

- 1 Dimensões e fatores de qualidade (Garvin, McCall, ISO 25010)
- 2 Processo x produto — e o Dilema da Qualidade
- 3 Técnicas de revisão: por que revisar e o que isso custa
- 4 Amplificação de defeitos — um exercício prático
- 5 Kahoot valendo nota (AC1)

01

Dimensões e Fatores de Qualidade

De visões a dimensões

Na Aula 2 vimos as cinco visões de Garvin sobre o que é qualidade. Ele foi além: propôs oito dimensões mais concretas, pensadas para tornar essas visões observáveis na prática.

As Oito Dimensões de Garvin (1/2)

- **Desempenho** — o software entrega todo o conteúdo, funções e recursos especificados, gerando valor ao usuário final?
- **Confiabilidade** — o software funciona sem falhas, está disponível quando necessário e entrega a função sem erros?
- **Características** — o software surpreende e encanta quem o usa pela primeira vez?
- **Conformidade** — o software segue os padrões e convenções de projeto e codificação da área?

As Oito Dimensões de Garvin (2/2)

- **Durabilidade** — o software pode ser corrigido ou modificado sem gerar efeitos colaterais indesejados?
- **Atendimento** — correções e mudanças podem ser feitas em tempo aceitável, com informação suficiente para quem dá suporte?
- **Estética** — o software tem elegância, um fluir único, uma "presença" difícil de quantificar mas evidente?
- **Qualidade percebida** — a reputação do fabricante influencia a percepção de qualidade — para o bem ou para o mal?

Pensem em um produto eletrônico ou aplicativo que vocês conhecem bem. Qual das oito dimensões é o ponto forte dele? E qual é o ponto fraco?

Conversem em duplas por 1 minuto — depois eu chamo só 2 ou 3 duplas para compartilhar.

Uma segunda proposta: McCall

McCall, Richards e Walters propuseram uma categorização diferente: fatores de qualidade organizados em torno de três grandes aspectos de um produto de software.

Fatores de McCall — Características Operacionais

- Correção — o quanto o programa atende à especificação e aos objetivos do cliente
- Confiabilidade — o quanto se pode esperar que ele funcione com a precisão exigida
- Eficiência — recursos computacionais exigidos para desempenhar a função
- Integridade — o quanto o acesso por pessoas não autorizadas pode ser controlado
- Usabilidade — esforço para aprender, operar e interpretar a saída do programa

Fatores de McCall — Mudança e Adaptação

- Facilidade de manutenção — esforço para localizar e corrigir um erro
- Flexibilidade — esforço para modificar um programa em operação
- Testabilidade — esforço para testar e garantir que a função desejada funciona
- Portabilidade — esforço para transferir o programa entre ambientes
- Reusabilidade — o quanto ele pode ser reaproveitado em outras aplicações
- Interoperabilidade — esforço para integrar o sistema a outro

O problema de medir isso tudo

Muitos desses fatores são difíceis — às vezes impossíveis — de medir diretamente. Isso abriu espaço para uma tentativa de padronização internacional.

Daqui nasce a família de normas ISO sobre qualidade de produto de software.

De olho no que já mudou

O material clássico sobre qualidade de software cita a norma ISO/IEC 9126, com seis características. Ela já foi substituída: hoje o padrão vigente é a ISO/IEC 25010, atualizada em 2023.

ISO/IEC 9126 → ISO/IEC 25010:2023

ISO/IEC 25010:2023 — Oito Características

- Adequação funcional
- Eficiência de desempenho
- Compatibilidade — nova em relação à 9126
- Usabilidade
- Confiabilidade
- Segurança — nova em relação à 9126
- Manutenibilidade
- Portabilidade

ENQUETE RÁPIDA

Nos projetos de disciplina de vocês, qual dessas características costuma ficar mais esquecida?



Segurança

Usabilidade

Manutenibilidade

Portabilidade

Para responder acesse o QR Code.

02

Processo x Produto — e o Dilema da Qualidade

Duas dimensões, um objetivo

É impossível obter software de qualidade com um processo de desenvolvimento frágil. Por isso, garantia da qualidade precisa olhar, ao mesmo tempo, para o produto tecnológico e para o processo que o gerou.

Qualidade do processo + qualidade do produto = qualidade do software.

Normas de Produto e de Processo

- Produto: ISO/IEC 25010 (características de qualidade) e ISO/IEC 25040 (avaliação — sucessora da antiga ISO 14598)
- Processo: ISO/IEC 12207 (ciclo de vida), ISO 90003 (ISO 9001 aplicada a software)
- Processo: ISO/IEC 33001 (avaliação de processos — sucessora do antigo SPICE/ISO 15504)
- Processo: CMMI e MPS.BR — os modelos que já vimos nas aulas anteriores

Qualidade do Processo

Essa dimensão exige estabelecer uma **cultura de não tolerância a erros**: procedimentos que avaliam a qualidade dos artefatos ao longo de todo o ciclo de desenvolvimento, permitindo encontrar defeitos cedo.

Qualidade do Produto

Essa dimensão é a mais visível: **toda empresa de software testa, de alguma forma, o que produz.** O objetivo é "estressar" telas e funcionalidades até garantir que o produto tecnológico entregue funciona como deveria.

O Dilema da Qualidade de Software

- Software de péssima qualidade — ninguém quer comprar
- Software perfeito, mas com esforço enorme de produção — custo tão alto que pode levar a empresa à falência

O Meio-Termo Mágico

Entre o software ruim e o software perfeito e inviável existe um **ponto de equilíbrio: esforço e complexidade de desenvolvimento** na medida do necessário para um software suficientemente bom.

ENQUETE RÁPIDA

É aceitável produzirmos software
"bom o suficiente"?



SIM

NÃO

Respondam pelo QR code

Depende do Contexto

- Uma pequena empresa entregando um sistema com erros — pode não ter uma segunda chance com o cliente
- Uma grande empresa entregando um sistema com erros — o nome da marca pode absorver o impacto
- Software embarcado com erros — pode gerar litígio, prejuízo grave e, em alguns casos, crime

Procedam com Cautela

"Bom o suficiente" não é um atalho universal para resolver problemas de qualidade. Pode funcionar — mas só em poucos casos e em um conjunto limitado de domínios de aplicação.

Ninguém quer software bom o suficiente e inútil.

O Custo da Qualidade — Três Categorias

- **Prevenção** — planejar, coordenar, treinar e desenvolver modelos completos de requisitos e projeto antes de codificar
- **Avaliação** — revisões técnicas, coleta de métricas, testes e depuração para entender a condição real do produto
- **Falhas** — tudo o que desapareceria se nenhum erro tivesse surgido, antes ou depois da entrega

Custos de Falha — Interna x Externa

Falhas internas (antes de entregar)

- Retrabalho para corrigir o erro
- Efeitos colaterais gerados pelo retrabalho
- Coleta de métricas para entender os modos de falha

Falhas externas (depois de entregar)

- Resolução de reclamações
- Devolução e substituição de produtos
- Suporte técnico por telefone ou e-mail
- Custos de garantia do produto

Alcançando a Qualidade de Software

Qualidade não aparece do nada. Ela é resultado de bom gerenciamento de projeto e prática consistente de engenharia de software — apoiados em quatro grandes atividades.

As Quatro Atividades

- Métodos de engenharia de software
- Técnicas de gerenciamento de projeto
- Controle da qualidade — reativo (já vimos na Aula 2)
- Garantia da qualidade — proativa (já vimos na Aula 2)

03

Técnicas de Revisão

Revisões Como Filtro

As revisões de software funcionam como um filtro na gestão de qualidade: são aplicadas em várias etapas do processo e **servem para revelar defeitos que podem ser eliminados antes de avançar.**

Nem de Menos, Nem de Mais

- Poucas revisões e o fluxo fica sujo.
- Revisões demais e o fluxo vira um gotejamento.
- A saída é usar métricas para saber quais revisões realmente funcionam — e eliminar do processo as que não funcionam.

Use dados para decidir onde revisar, não só instinto.

Por Que Revisar?

O trabalho técnico precisa de revisão pela mesma razão que o lápis precisa de borracha: **errar é humano**. E boa parte dos nossos próprios erros escapa mais facilmente de nós do que de quem olha de fora.

O Que Uma Revisão Faz

- Aponta aperfeiçoamentos necessários no produto de uma pessoa ou de uma equipe
- Confirma as partes do produto em que nenhum aperfeiçoamento é necessário
- Produz trabalho técnico de qualidade mais uniforme — e mais previsível

SERÁ QUE EXISTE DIFERENÇA?

Erro x Defeito

Um Detalhe Importante

- Na prática, a comunidade de engenharia de software trata erro, defeito, falha e bug como sinônimos.
- O que realmente importa é encontrar o problema antes que o cliente o encontre, não o rótulo que se dá a ele.

O Esforço Poupado

Corrigir um erro de requisitos numa revisão custa cerca de 6 homens/hora. O mesmo erro, encontrado só nos testes, custa em média 45 homens/hora — uma economia de 30 homens/hora por erro encontrado cedo.

Em 22 erros encontrados numa revisão: cerca de 660 homens/hora de esforço de teste poupados.

Métricas de Revisão — Um Primeiro Contato

- Esforço de preparação (Ep) e esforço de avaliação (Ea) — homens/hora antes e durante a reunião de revisão
- Esforço de reformulação (Re) — homens/hora para corrigir os erros revelados
- Tamanho do artefato (TAS) — nº de páginas, linhas de código ou modelos revisados
- Densidade de erros = total de erros ÷ tamanho do artefato

Vamos Aprofundar na Próxima Aula

Essas métricas — e como usá-las para prever quantos erros um novo documento provavelmente tem, mesmo sem revisá-lo — são o tema central da Técnicas de Revisão Parte II.

Eficácia das Revisões — Os Clássicos

10:1

Retorno sobre investimento em inspeções reportado
pela HP

-10x

Redução no custo total de erros de software reportada
pela AT&T

AT&T também reportou +14% de produtividade com o uso de inspeções técnicas.

70–90%

**dos defeitos são encontrados em revisões de código bem conduzidas
(SmartBear / Cisco)**

O ideal é revisar abaixo de 500 linhas de código por hora. E revisões leves — sem reunião formal — levam menos de 20% do tempo de uma inspeção formal e encontram praticamente os mesmos erros.

Revisões Não Gastam Tempo

O esforço de revisar aumenta no início do desenvolvimento de um módulo. Mas esse investimento inicial rende dividendos: o esforço de teste e correção cai, e a data de entrega chega mais cedo.

As revisões não gastam tempo — elas poupam.

Nos projetos de disciplina que vocês desenvolveram ao longo do curso de Ciência da computação: em que momento uma revisão de código, requisitos ou documento teria evitado retrabalho depois? Deem um exemplo concreto.

Conversem em duplas por 1 minuto — depois eu chamo só 2 ou 3 duplas para compartilhar.

Conclusão

- As dimensões de Garvin, os fatores de McCall e a ISO/IEC 25010 são três formas complementares de operacionalizar "qualidade"
- "Bom o suficiente" não é atalho universal — depende do domínio, do porte da empresa e do risco envolvido
- Revisões técnicas são um filtro: encontrar um erro cedo custa uma fração do que custa encontrá-lo depois
- Erro, defeito, falha e bug são sinônimos na prática — o que importa é achar o problema antes do cliente

Próxima Aula

08/09/2026 — Unidade I: Técnicas de Revisão (Parte II) — métricas de revisão na prática.

Tragam o exemplo que vocês compartilharam hoje sobre revisão evitando retrabalho!

KAHOOT · GAMIFICAÇÃO

Vamos testar o que aprendemos?

Questionário (AC1) · Kahoot

Abram o celular agora — vale nota

**Revisar não é
perder tempo!**

Obrigada! Dúvidas?